LAB SHEET 4

Experiment Name : Frequency Domain Image Filtering using Fourier Transform

Steps

1. Import the required libraries and load a grayscale image for processing.
2. Compute the Discrete Fourier Transform (DFT) of the image using OpenCV or NumPy.
3. Shift the zero-frequency component to the center of the frequency spectrum for visualization.
4. Display and analyze the magnitude spectrum of the transformed image.
5. Design and apply a Low-Pass Frequency Filter to suppress high-frequency components and reduce image noise.
6. Design and apply a High-Pass Frequency Filter to suppress low-frequency components and enhance edges and fine details.
7. Perform the Inverse Fourier Transform (IDFT) to reconstruct the filtered images in the spatial domain.
8. Compare the original image, frequency spectrum, low-pass filtered image, and highpass filtered image.
9. Analyze the impact of frequency domain filtering on image smoothness, edge enhancement, and noise removal.
10. Document the observations and discuss the advantages of frequency domain filtering in practical computer vision applications.

Solution: **main.py:**

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os
image_path = "image.jpg"
if not os.path.exists(image_path):
print("ERROR: image.jpg was not found!")
print("Put image.jpg in the same folder as experiment4.py")
exit()
img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
if img is None:
print("ERROR: Image could not be loaded.")
exit()
print("Image loaded successfully!")
print("Image size:", img.shape)
dft = cv2.dft(
np.float32(img),
flags=cv2.DFT_COMPLEX_OUTPUT
)
print("DFT calculated successfully!")
dft_shift = np.fft.fftshift(dft)
print("Frequency shifted successfully!")
```

```
magnitude = cv2.magnitude(
dft_shift[:, :, 0],
dft_shift[:, :, 1]
)
magnitude_spectrum = 20 * np.log(
magnitude + 1
)
spectrum_display = cv2.normalize(
magnitude_spectrum,
None,
0,
255,
cv2.NORM_MINMAX
).astype(np.uint8)
cv2.imwrite(
"frequency_spectrum.jpg",
spectrum_display
)
print("Frequency spectrum saved!")
rows, cols = img.shape
crow = rows // 2
ccol = cols // 2
radius = 50
low_pass_mask = np.zeros(
(rows, cols, 2),
np.float32
)
cv2.circle(
low_pass_mask,
(ccol, crow),
radius,
(1, 1),
-1
)
low_pass_dft = dft_shift * low_pass_mask
low_pass_ishift = np.fft.ifftshift(
low_pass_dft
)
low_pass_result = cv2.idft(
low_pass_ishift
)
low_pass_result = cv2.magnitude(
low_pass_result[:, :, 0],
low_pass_result[:, :, 1]
)
low_pass_result = cv2.normalize(
low_pass_result,
None,
```

```
0,
255,
cv2.NORM_MINMAX
).astype(np.uint8)
cv2.imwrite(
"low_pass_result.jpg",
low_pass_result
)
print("Low-pass result saved!")
high_pass_mask = np.ones(
(rows, cols, 2),
np.float32
)
cv2.circle(
high_pass_mask,
(ccol, crow),
radius,
(0, 0),
-1
)
high_pass_dft = dft_shift * high_pass_mask
high_pass_ishift = np.fft.ifftshift(
high_pass_dft
)
high_pass_result = cv2.idft(
high_pass_ishift
)
high_pass_result = cv2.magnitude(
high_pass_result[:, :, 0],
high_pass_result[:, :, 1]
)
high_pass_result = cv2.normalize(
high_pass_result,
None,
0,
255,
cv2.NORM_MINMAX
).astype(np.uint8)
cv2.imwrite(
"high_pass_result.jpg",
high_pass_result
)
print("High-pass result saved!")
plt.figure(figsize=(12, 8))
plt.subplot(2, 2, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Image")
plt.axis("off")
```

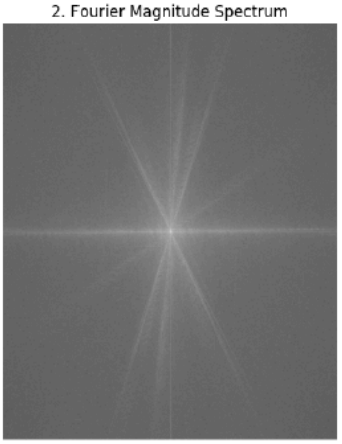
```
plt.subplot(2, 2, 2)
plt.imshow(magnitude_spectrum, cmap="gray")
plt.title("Frequency Spectrum")
plt.axis("off")
plt.subplot(2, 2, 3)
plt.imshow(low_pass_result, cmap="gray")
plt.title("Low-Pass Filter")
plt.axis("off")
plt.subplot(2, 2, 4)
plt.imshow(high_pass_result, cmap="gray")
plt.title("High-Pass Filter")
plt.axis("off")
plt.tight_layout()
plt.savefig(
"experiment4_output.png",
dpi=300,
bbox_inches="tight"
)
plt.close()
print()
print("============================================")
print("EXPERIMENT COMPLETED SUCCESSFULLY!")
print("============================================")
print("Output files created:")
print("1. frequency_spectrum.jpg")
print("2. low_pass_result.jpg")
print("3. high_pass_result.jpg")
print("4. experiment4_output.png")
print("============================================")
```

output.png:

1. Original Grayscale Image



2. Fourier Magnitude Spectrum



3. Low-Pass Filter (Radius = 204)



4. High-Pass Filter (Radius = 204)

